

BÖLÜM 1: GİRİŞ VE TEORİK MODEL

1. Problem Tanımı ve Amaç

Bu proje kapsamında, modern bilgisayar mimarilerinin ve işlemcilerin (CPU) aritmetik mantık birimlerinde (ALU) kullanılan temel aritmetik yöntemlerden biri olan "Kaydır ve Topla" (Shift and Add) mantığı ele alınmıştır. Bilgisayarlar donanım seviyesinde çarpma işlemini doğrudan tek bir hamlede yapamazlar; bunun yerine bitleri sola doğru kaydırarak toplama havuzuna eklerler.

Bu projenin amacı, söz konusu donanımsal çarpma algoritmasını soyut matematiksel bir model olan Tek Bantlı Deterministik Turing Makinesi (Single-Tape DTM) üzerinde simüle etmektir. Süreç boyunca, şerit üzerindeki verilerin bütünlüğünü korumak, girdi sınırlarını belirlemek ve sonlu durumlar (states) arasında kararlı geçişler tasarlamak hedeflenmiştir.

2. Turing Makinesi Teorik Modeli

Geliştirilen simülatör, matematiksel olarak $M = (Q, \Sigma, \Gamma, \delta, q_{\text{basla}}, q_{\text{kabul}})$ bileşenlerinden oluşan 6'lı bir küme olarak modellenmiştir:

- Durum Kümesi (Q):** Makinenin algoritmayı yürütürken geçiş yaptığı tüm mantıksal durumları içerir:
- $Q = \{q_{\text{basla}}, q_{\text{carpan_oku}}, q_{\text{offset_kaydir}}, q_{\text{en_saga_0_yaz}}, q_{\text{toplama_modu}}, q_{\text{carpilan_bit_sec}}, q_{\text{tasi_0}}, q_{\text{tasi_1}}, q_{\text{ekle_0}}, q_{\text{ekle_1}}, q_{\text{elde_modu}}, q_{\text{tasima_bitti}}, q_{\text{carpilan_restore}}, q_{\text{tur_sonu_kaydir}}, q_{\text{eve_don}}, q_{\text{temizlik}}, q_{\text{sonu_ara}}, q_{\text{fazlalik_kirp}}, q_{\text{kabul}}\}$
- Giriş Alfabeti (Σ):** Kullanıcının bant üzerinde tanımlayabileceği ham karakter kümesidir: $\Sigma = \{0, 1, *, =\}$
- Bant Alfabeti (Γ):** İşlem sırasında makinenin hafıza yönetimi ve işaretleme için kullandığı tüm sembolleri kapsar: $\Gamma = \{0, 1, *, =, U, V, X, Y, B\}$
- Başlangıç Durumu:** q_{basla}
- Kabul Durumu:** q_{kabul}

3. Operand Ayırıştırma Yaklaşımı (Kritik Tasarım)

Ödev föyünde zorunlu tutulan "*Operandların bant üzerinde doğru ayrıştırılması*" gereksinimi, projede "Dinamik Maskeleyme ve Geri Yükleme" (Dynamic Masking & Restoration) yaklaşımıyla çözülmüştür. Makine tek bir şerit üzerinde çalıştığı için sol taraftaki sayı (çarpılan) ile sağ taraftaki sayı (çarpan) işlem anında birbirine karışma riski taşır. Bunun önüne geçmek için şu mimari uygulanmıştır:

- Çarpanın (Sağdaki Sayı) Yönetimi:** * işaretinin sağındaki çarpan bitleri sağdan sola doğru taranır. İşlenen bitler 0 ise U sembolüyle, 1 ise V sembolüyle kalıcı olarak maskelenir. Bu sayede makine sağa her döndüğünde sıradaki işlenmemiş biti hatasız tespit eder.
- Çarpılanın (Soldaki Sayı) Yönetimi:** Çarpan biti 1 olduğunda toplama moduna girilir. Sol taraftaki çarpılan sayının bitleri sonuç alanına taşınırken geçici olarak 0 $\rightarrow$ X ve 1 $\rightarrow$ Y sembollerine dönüştürülür.
- Hata Önleme ve Bellek Koruma:** Bir toplama turu bittiğinde makine $q_{\text{carpilan_restore}}$ durumuna geçer. Bu durumda, geçici olarak değiştirilen X ve Y sembolleri tekrar 0 ve 1 haline getirilir. Ayrıca, ikili sayı sisteminde ortada bulunan yalın 0 bitlerinin (Örn: 101_2 sayısındaki ortadaki sıfır) yapısı korunarak üzerinden atlanır. Böylece iki operandın bellek sınırları işlem boyunca asla ihlal edilmez.

BÖLÜM 2: DURUM AÇIKLAMALARI VE GEÇİŞ MANTIĞI

1. Durumların (States) Fonksiyonel Açıklamaları

Turing Makinesinin kararlı çalışmasını sağlayan algoritma, işlevlerine göre 4 ana faza bölünmüş durumlar içermektedir:

Faz 1: Sağdaki Sayıyı (Çarpan) Tespit Etme ve Tarama

- **q_basla:** Makinenin ilk durumudur. Şeridin en solundan başlayarak sağa doğru ilerler ve sonuç alanının başlangıcı olan = sembolünü arar. Yol üstündeki sayıları ve maskeleri atlar.
- **q_carpan_oku:** = sembolünün soluna (yani çarpan sayısına) geçer. Sağdan sola doğru tarama yaparak henüz işlenmemiş ilk ham biti arar. Eğer 1 bulursa toplama modunu tetikler, 0 bulursa ofset modunu tetikler. Tüm çarpan bitleri bittiğinde ve doğrudan * işaretine çarptığında çarpma işleminin bittiğini anlar ve temizlik fazına geçer.

Faz 2: Çarpan Biti '0' İse (Ofset Kaydırma)

- **q_ofset_kaydir:** Okunan çarpan biti 0 olduğunda, matematiksel olarak herhangi bir sayı eklenmeyeceği, sadece basamak kaydırılacağı için şeridin en solundaki boşluğa (B) kadar sola doğru gider.
- **q_en_saga_0_yaz:** Şeridin en solundan kalkıp en sağına kadar gider ve sonuç alanının sonuna basamak koruma amaçlı bir 0 ekler. Ardından başa dönmek için kafayı sola yönlendirir.

Faz 3: Çarpan Biti '1' İse (Kopyala ve İkili Topla)

- **q_toplama_modu:** Okunan çarpan biti 1 olduğunda tetiklenir. Çarpılan (soldaki) sayının bitlerine ulaşmak için * işaretinin soluna doğru hızla kayar.
- **q_carpilan_bit_sec:** Çarpılan sayının sağdan sola doğru işlenmemiş ilk bitini seçer. Seçtiği biti unutmamak için 0 -> X veya 1 -> Y olarak maskeler. Eğer tüm bitler X/Y olmuşsa ve sola doğru boşluğa (B) çarptıysa, mevcut turun bittiğini anlar ve geri yükleme durumuna geçer.
- **q_tasi_0 / q_tasi_1:** Seçilen bit hafızaya alınır ve sağ taraftaki sonuç (toplama) alanına taşınmak üzere tüm şerit boyunca sağa doğru koridor şeklinde akar.
- **q_ekle_0 / q_ekle_1:** = sembolünün sağındaki sonuç alanında ikili matematik kurallarına göre hücreye ekleme yapar. Eğer q_ekle_1 durumundayken şeritte zaten 1 varsa, ikili sistem kuralı gereği (1+1=0, elde 1 var) hücreye 0 yazar ve elde moduna geçer.
- **q_elde_modu:** Matematiksel toplama işlemindeki "Elde Var 1" (Carry) zincirini yönetir. Sola doğru giderek ilk uygun boşluğa veya 0 bitine eldeki 1 değerini bırakır.
- **q_tasima_bitti:** Tek bir bitin kopyalanıp toplama alanına başarıyla eklenmesinin ardından, çarpılan sayının bir sonraki bitini seçmek üzere şeridin soluna geri döner.
- **q_carpilan_restore:** Çarpan biti 1 olduğu için yapılan kopyalama turu tamamlandığında, çarpılan sayının üzerindeki tüm geçici X ve Y maskelerini tekrar 0 ve 1 haline getirir. Bu esnada 101_2 gibi sayıların ortasında bulunan yalın 0 bitlerini güvenle korur.
- **q_tur_sonu_kaydir:** Bir turun bittiğini onaylamak amacıyla sonuç alanının en sağına basamak kaydırma sıfırı ekler.
- **q_eve_don:** Yapılan tüm alt işlemlerden sonra makineyi bir sonraki ana çarpan bitini okuması için en başa, yani q_basla fazına güvenle geri uçurur.

Faz 4: Temizlik ve Kapanış

- **q_temizlik:** Çarpma tamamen bittiğinde şeritte kalan tüm U ve V maskelerini kaldırarak orijinal 0 ve 1 formuna dönüştürür.
- **q_sonu_ara & q_fazlalik_kirp:** Algoritmanın doğası gereği sonuç alanının solunda kalan matematiksel olarak anlamsız fazlalık sıfırları (0) şeritten budar (silip B yapar) ve makineyi durdurur.
- **q_kabul:** Makinenin çarpma işlemini hatasız bitirdiği nihai durma durumudur.

2. Durum Geçiş Tablosu (State Transition Table)

Mevcut Durum (Qn)	Okunan Sembol (Γ)	Yazılan Sembol (Γ)	Hareket Yönü (D)	Sonraki Durum (Qn+1)	Açıklama
q_basla	0 / 1 / * / U / V	Değişmez	R	q_basla	= işaretine kadar sağa tarama
q_basla	=	=	L	q_carpan_oku	Sağdaki sayıya giriş yap
q_carpan_oku	U / V	Değişmez	L	q_carpan_oku	İşlenmiş bitleri atla
q_carpan_oku	0	U	L	q_ofset_kaydir	Çarpan 0 ise maskele ve kaydır
q_carpan_oku	1	V	L	q_toplama_modu	Çarpan 1 ise maskele ve topla
q_carpan_oku	*	*	R	q_temizlik	İşlem bitti, temizliğe geç
q_carpilan_bit_sec	0	X	R	q_tasi_0	Çarpılan bitini hafızaya al
q_carpilan_bit_sec	1	Y	R	q_tasi_1	Çarpılan bitini hafızaya al
q_carpilan_bit_sec	B	B	R	q_carpilan_restore	Tur bitti, maskeleri

					çözmeye git
q_ekle_1	1	0	L	q_elde_mod u	\$1+1=0\$ (Elde var 1 tetikleme)
q_elde_mod u	1	0	L	q_elde_mod u	Eldeyi sola taşımaya devam et
q_elde_mod u	0 / B	1	L	q_tasima_bitti	Eldeyi uygun boşluğa yaz
q_carpilan_restore	X	0	R	q_carpilan_restore	Geçici maskeyi sıfıra geri döndür
q_carpilan_restore	Y	1	R	q_carpilan_restore	Geçici maskeyi bire geri döndür
q_carpilan_restore	0	0	R	q_carpilan_restore	Yapısal sıfırları

BÖLÜM 3: GİRDİ-ÇIKTI ÖRNEKLERİ VE DEĞERLENDİRME

1. Girdi-Çıktı ve Doğrulama Örnekleri

Geliştirilen simülatörün kararlılığını, operand ayrıştırma mekanizmasını ve sınır durumlarındaki (edge cases) başarısını test etmek amacıyla 5 farklı senaryo çalıştırılmıştır. Terimlerden elde edilen test sonuçları, ikili ve onluk sistem doğrulamalarıyla birlikte aşağıda sunulmuştur:

Test 1: 10x10

Test 2: 10x101

Test 3: 11x10

Test 4: 11x11

Test 5: 101x10

2. Sonuç, Hata Analizi ve Değerlendirme

Bu proje çalışmasında, tek bir sonsuz şerit ve sınırlı sayıda durum (state) kısıtı altında, karmaşık bir aritmetik işlem olan ikili çarpma başarıyla modellenmiştir. Geliştirilen

Deterministik Turing Makinesi (DTM), modern işlemcilerin mikro mimari düzeyinde gerçekleştirdiği "kaydır ve topla" mantığını tam bir tutarlılıkla yansıtmaktadır.